Punteros

Los punteros son una de las herramientas más poderosas con las que cuenta C. Este lenguaje es considerado de bajo nivel y el manejo de los punteros es en parte lo que determina este bajo nivel de programación ya nos permite acceder y manipular los contenidos de direcciones de memoria. Un puntero es una variable que almacena la dirección de memoria de otra variable. Un puntero por si solo si no apunta a nada no almacena ni contiene información.

Sintaxis y definición de un puntero

```
int *numero;
```

Declarando un tipo de dato y un nombre de variable anteponiendo el asterisco, estamos diciendo que declaramos un puntero del tipo entero llamado número. ¿Qué significa esto? Que esta variable "***número"*** no contiene en sí un valor entero sino una dirección de memoria en donde sí está guardado un entero. Tal como dijimos antes un puntero de por sí solo si no apunta a nada no contiene nada. En este punto declaramos un puntero o apuntador que va a apuntar a una dirección de memoria que contenga enteros. Sigamos viendo:

```
int *numero;
int valor=8;
numero=&valor;

printf("La variable valor tiene el numero %d\n",valor);
printf("El puntero numero apunta a la variable y tiene el valor %d",*numero);
```

Si ejecutamos el código anterior en ambos casos vemos por pantalla el número 8. Esto quiere decir que el puntero ***numero*** apunta a la dirección de memoria donde esta guardada la variable ***valor***. Por lo tanto, si luego de lo anterior modificamos el contenido de la variable ***valor*** y volvemos a mostrar ***número***, también vamos a ver el cambio reflejado.

```
int *numero;
int valor=8;
numero=&valor;

printf("La variable valor tiene el numero %d\n",valor);
printf("El puntero numero apunta a la variable y tiene el valor %d\n",*numero);

valor++;
printf("Ahora numero muestra el numero %d",*numero);
```

Aumentamos valor y al mostrar el contenido de numero vemos 9 por pantalla. Debemos tener en cuenta que si queremos mostrar un entero a través del puntero necesitamos poner *numero, ya que de esta manera accedemos al contenido de la memoria, por lo tanto, al entero almacenado. Si solo ponemos número estamos mostrando o accediendo a la dirección de memoria.

En la línea ***numero=&valor*** estamos asignando la dirección de memoria. El operador ampersand "&" es la herramienta que utilizamos para obtener la dirección de memoria de una variable. De esta manera en esa línea establecemos hacia donde apunta nuestro puntero.

Variables por valor y por referencia

La utilización de punteros nos sirve para poder manipular el contenido de una variable en forma indirecta, veamos un ejemplo de cómo intercambiar dos valores a través de una función:

```
void intercambiar(int a,int b)
{
    int c=a;
    a=b;
    b=c;
}

int main()
{
    int x=8;
    int y=3;
    printf("El valor de x es %d y el de y es %d",x,y);
    intercambiar(x,y);
    printf("El valor de x es %d y el de y es %d",x,y);
    return 0;
}
```

Si ejecutamos el código anterior nuestro propósito sería que se intercambien los valores pasados a la función, pero cuando vemos los resultados por pantalla esto no es lo que sucede. Cuando nostros realizamos la llamada a la función estamos pasando variable por valor, osea una copia de ellas y el intercambio se produce únicamente dentro de las copias de la función. Al finalizar la función las variables x e y no sufrieron cambios. Para poder solucionar estos utilizaremos punteros de tal manera de pasar una referencia de las variables y así en vez de intercambiar una copia, intercambiamos el contenido real de las variables.

```
void intercambiar(int *a,int *b)
{
    int c=*a;
    *a=*b;
    *b=c;
}

int main()
{
    int x=8;
    int y=3;
    printf("El valor de x es %d y el de y es %d\n",x,y);
    intercambiar(&x,&y);
    printf("El valor de x es %d y el de y es %d",x,y);
    return 0;
}
```

Punteros y arreglos

Una forma de trabajar con arreglos o vectores en c es utilizando punteros. En definición un vector en C es realmente un puntero. Por más que esté definido como una variable estática de por si internamente esta creado en memoria como un puntero, veamos el siguiente ejemplo:

```
void carga_vector(int * vec)
{
    for(int i=0;i<tam;i++)
     vec[i]=i;
}

void recorre_vector(int vec[])
{
     for(int i=0;i<tam;i++)
         printf("%d\n",vec[i]);
}

 int main()
 {
     int vec[tam];
     carga_vector(vec);
     recorre_vector(vec);
     return 0;
 }
```

Tenemos dos funciones y un solo vector, en la función de carga de datos lo recibimos como puntero y en la de recorrer está como vector estático. Al realizar la llamada para C es indiferente, en cualquiera de los casos lo toma como vector.

Llamadas a memoria

Habíamos visto antes que no podemos utilizar un puntero si este no apunta a ninguna dirección de memoria. En ejemplos anteriores vimos como referenciábamos el puntero a otra variable, pero también tenemos la posibilidad de referenciar el puntero a un espacio de memoria creado particularmente, para eso vamos a hacer uso de la función ***malloc***:

```
int main()
{
    int *puntero;
    puntero=(int*)malloc(sizeof(int));
    *puntero=8;
    printf("%d",*puntero);
    free(puntero);
    return 0;
}
```

La función ***malloc***, lo que hacer es una solicitud de memoria al sistema. La sentencia proviene de ***memory allocation*** y lo que hacemos es destinar una porción de memoria para uso como variable. En este caso asignamos esa memoria a un puntero de entero ***(int*)*** y decimos que ese puntero va a tener el tamaño de un entero ***sizeof(int).*** Esta memoria no es gestionada por el sistema, por lo tanto, nos tenemos que hacer cargo de su uso y liberación cuando no la utilizamos mas. Si tenemos un programa grande y constantemente hacemos solicitudes de memoria sin liberar la que no está en uso, corremos el riesgo de colapsar el sistema. Esta liberación de memoria la realizamos con la función ***free(puntero),*** con esto estamos devolviendo la disponibilidad de esa memoria al sistema. Esta es una de las herramientas más poderosas que tiene C ya que pasamos por encima del sistema operativo anfitrión pudiendo gestionar programas con uso muy bajo de memoria.

Función realloc

Cuando hacemos una reserva de memoria con ***malloc*** podemos correr el riesgo de necesitar más memoria adicional o por el contrario un bloque de memoria ha quedado grande y necesitamos achicarlo, para eso la función ***realloc*** (re allocation memory) se utiliza para cambiar el tamaño de un bloque de memoria previamente asignado por ***malloc*** o ***calloc***, conservando el contenido hasta el menor de los tamaños (original o nuevo).

Sintaxis

```
void* realloc(void* ptr, size_t new_size);
```

- ptr: Puntero a la memoria previamente asignada. Si es NULL, se comporta como malloc.
- new_size: Nuevo tamaño deseado en bytes.
- retorno: Un puntero genérico (void*) al nuevo bloque de memoria. Si no puede asignar memoria, devuelve NULL.

Función calloc

La función ***calloc*** es similar a ***malloc***, pero con una diferencia importante: asigna memoria para múltiples elementos e inicializa todos los bytes a cero.

Sintaxis

```
void* calloc(size_t num, size_t size);
```

- num: El número de elementos a asignar.
- size: El tamaño de cada elemento.
- retorno: Un puntero genérico (void*) al bloque de memoria asignado.

Operador free

Cada vez que reservamos memoria al sistema, esta memoria queda ocupada hasta que dejemos de utilizarla y la liberemos. En variables estáticas esta memoria se libera automáticamente, pero utilizando punteros debemos ocuparnos de esta tarea:

```
int main()
{
   int *numero=(int*)malloc(sizeof(int));
   *numero=3;
   free(numero);
}
```